

M1S4 | Async Programming

Main Concepts

Multithreading

multiple threads inside one process, shared memory and resources

Multiprocessing

multiple processes, separate memory and resources

Asynchronous Programming

event loop, pipeline analogue

Multithreading & GIL

better for I/O-bound tasks

GIL reasons:

- memory safety
- realization CPython simplification
- C-extensions integrations

```
import threading
import time

def worker(name):
    for i in range(3):
        print(name, i)
        time.sleep(0.5)

t1 = threading.Thread(target=worker, args=("A",))
t2 = threading.Thread(target=worker, args=("B",))

t1.start()
t2.start()

t1.join()
t2.join()
```


Multiprocessing

better for CPU-bound tasks

```
1  from multiprocessing import Process
2  import time, os
3
4
5  def worker(name):
6      print(f"Start {name}, pid={os.getpid()}")
7      time.sleep(1)
8      print(f"End {name}, pid={os.getpid()}")
9
10
11  if __name__ == "__main__":
12      p1 = Process(target=worker, args=("A",))
13      p2 = Process(target=worker, args=("B",))
14
15      p1.start()
16      p2.start()
17
18      p1.join()
19      p2.join()
20
21      print("Done")
22
```

Async Programming

better for I/O bound tasks, uses Event Loop

```
import asyncio
import time

async def worker(name, delay):
    print(f"{time.strftime('%X')} start {name}")
    await asyncio.sleep(delay)
    print(f"{time.strftime('%X')} end {name}")

async def main():
    task1 = asyncio.create_task(worker("A", 2))
    task2 = asyncio.create_task(worker("B", 2))

    await task1
    await task2

await main()
```

Synchronization Primitives

Lock

only one thread can handle it

Semaphore

bound the number of
tsimultaneously working threads

Event

threads “communication”
mechanism

What's what for?

Feature	Multithreadnig	Multiprocessing	Async Programming
Parallelism	Limited(by GIL)	Fulli parallel on multicore	Cooperative multitasking
Context Switching Overhead	Low	High	Lowest
Shared State	Shared memory	Separate process memory	Shared memory
Synchronization	Locks, Semaphores, etc.	IPC(message passing)	Async/await, events
Suitability	I/O or CPU-bound	CPU intensive parallel processing	I/O-intensive tasks